WEEK 2 · LESSON 8 OF 24 · HANDOUT

Optimisers: SGD, Momentum and Adam

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=ErnWZxJovaM — open the lesson online to watch inside the course.

🎓 **Go deeper:** Harvard CS50 AI, Lecture 5: Neural Networks — https://www.youtube.com/watch?v=mFZazxxCKbw

🎓 **Go deeper:** freeCodeCamp: PyTorch for Deep Learning & Machine Learning — full course — https://www.youtube.com/watch?v=V_xro1bcAuA

Your goal in this lesson: Explain how SGD, momentum and Adam update weights, compare them on the same model, and choose a learning rate and a learning-rate schedule.

The optimiser decides how to step

All optimisers use the gradient. They differ in **how** they turn it into a step.

Optimiser	Idea	Everyday picture
SGD	Step = learning rate × gradient	Walking downhill one careful step at a time
SGD + momentum	Keep some speed from previous steps	A ball rolling downhill: it speeds up on long slopes and rolls over small bumps
Adam	Momentum + a separate, automatic step size for every weight	A hiker who takes short steps on rough ground and long steps on smooth ground

Adam (2014) is the most popular default: it works well with little tuning, usually with a learning rate around **0.001**.

A fair comparison

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X, y = digits.data / 16.0, digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0,
stratify=y)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2,
                                                  random_state=0, stratify=y_train)
to_x = lambda a: torch.tensor(a, dtype=torch.float32)
to_y = lambda a: torch.tensor(a, dtype=torch.long)
Xtr, Xva, Xte = to_x(X_train), to_x(X_val), to_x(X_test)
ytr, yva, yte = to_y(y_train), to_y(y_val), to_y(y_test)
print(len(Xtr), "train,", len(Xva), "validation,", len(Xte), "test")
```

```
def make_model(seed=0):
    torch.manual_seed(seed)
    return nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 10))

def train(optimizer_fn, epochs=20):
    model = make_model()
    opt = optimizer_fn(model.parameters())
    val_losses = []
    g = torch.Generator().manual_seed(0)          # same batches for every optimiser
    for epoch in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr), generator=g)
        for start in range(0, len(perm), 32):
            idx = perm[start:start + 32]
            opt.zero_grad()
            F.cross_entropy(model(Xtr[idx]), ytr[idx]).backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            val_losses.append(F.cross_entropy(model(Xva), yva).item())
    with torch.no_grad():
        acc = (model(Xva).argmax(1) == yva).float().mean().item()
    return val_losses, acc

runs = {
    "SGD lr=0.01": lambda p: torch.optim.SGD(p, lr=0.01),
    "SGD lr=0.1": lambda p: torch.optim.SGD(p, lr=0.1),
    "SGD + momentum": lambda p: torch.optim.SGD(p, lr=0.01, momentum=0.9),
    "Adam lr=0.001": lambda p: torch.optim.Adam(p, lr=0.001),
    "Adam lr=0.1": lambda p: torch.optim.Adam(p, lr=0.1),
}
for name, fn in runs.items():
    losses, acc = train(fn)
    plt.plot(losses, label=name)
    print(f"{name:15s} final validation loss {losses[-1]:.3f}  accuracy {acc:.3f}")
plt.xlabel("epoch")
plt.ylabel("validation loss")
plt.legend()
plt.show()
```

What you should see:

- **SGD with lr = 0.01** crawls: after 20 epochs it is still far from done (validation accuracy around 0.85).
- **Momentum** with the same small learning rate is many times faster.
- **SGD with lr = 0.1** and **Adam with lr = 0.001** both train well (about 0.94–0.96).
- **Adam with lr = 0.1** is too aggressive: its loss jumps around and ends worse.

The learning rate matters more than the choice of optimiser. Too small wastes time; too big makes training unstable or makes it explode.

How momentum works (in one line)

> *velocity = 0.9 × velocity + gradient, weight = weight − learning rate × velocity*

Gradients that keep pointing the same way add up; gradients that flip back and forth cancel out.

How to choose a learning rate

1. Start with the default: Adam 0.001, or SGD with momentum 0.01–0.1.
2. Try values 10 times apart (0.0001, 0.001, 0.01) and compare validation loss after a few epochs.
3. If the loss goes up or becomes `nan`, the learning rate is too big.

Learning-rate schedules

Big steps help early; small steps help to settle at the end. A **schedule** lowers the learning rate during training:

```
model = make_model()
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=20)
rates = []
for epoch in range(20):
    # ... one epoch of training would go here ...
    rates.append(opt.param_groups[0]["lr"])
    opt.step()          # (an optimiser step must come before the scheduler step)
    scheduler.step()    # once per epoch
plt.plot(rates)
plt.title("Cosine learning-rate schedule")
plt.xlabel("epoch")
plt.show()
```

The cosine schedule glides smoothly from 0.1 down to almost 0. `StepLR` (divide by 10 every few epochs) and `OneCycleLR` (warm up, then cool down) are other common choices.

Weight decay and AdamW

Optimisers can also gently pull weights towards zero every step — **weight decay**, the deep-learning version of Ridge regularisation. With Adam, use `torch.optim.AdamW(params, lr=1e-3, weight_decay=0.01)`, which applies it correctly. More in the next lesson.

Key terms

- **SGD** — stochastic gradient descent on mini-batches.
- **Momentum** — keeps a running "velocity" so steps build up speed.
- **Adam / AdamW** — adaptive step size for every weight; AdamW adds proper weight decay.
- **Learning-rate schedule** — lowers the learning rate during training.
- **Diverging** — the loss grows instead of shrinking, often because the learning rate is too big.

Remember this: Adam with a learning rate near 0.001 is a strong default; the learning rate is the single most important setting, and a schedule helps to finish training well.

Name: _____ Date: _____

Activity

Task 1

Add `"Adam lr=0.01"` to the comparison. Where does it land?

Task 2

Use `torch.optim.SGD(p, lr=0.1, momentum=0.9)`. Is it better or worse than plain SGD at 0.1?

Task 3

Put the cosine scheduler into a real training run with SGD + momentum and compare the final validation accuracy.

Task 4

Find the largest Adam learning rate that still trains smoothly on the digits.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does momentum add to SGD?

   A. A new loss
   B. Random noise
   C. More layers
   D. A running velocity so consistent gradients build up speed

2. What is special about Adam?

   A. It adapts the step size for every weight automatically
   B. It removes the learning rate
   C. It only works on CPUs
   D. It needs no gradients

3. What is a common default learning rate for Adam?

   A. 0.0000001
   B. 0.001
   C. 1
   D. 10

4. The training loss suddenly grows and becomes nan. What is the most likely cause?

   A. The learning rate is too large
   B. The batch is too small
   C. The data is too clean
   D. Too few epochs

5. What does a learning-rate schedule do?

   A. Changes the learning rate during training, usually lowering it
   B. Deletes old weights
   C. Changes the model architecture
   D. Shuffles the data

6. Which optimiser applies weight decay correctly for Adam-style training?

   A. Softmax
   B. SGD
   C. AdamW
   D. Momentum

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.